BÁO CÁO THỰC HÀNH TUẦN 5

`Họ và tên: Phạm Ngọc Hào`

`MSSV: 23110146`

1. Tóm tắt mã nguồn tự triển khai (Vertical Apriori / ECLAT)

Phần mã nguồn tự viết triển khai thuật toán **ECLAT** (Equivalence Class Transformation) dựa trên định dạng dữ liệu dọc (Vertical Data Format).

Các thành phần chính:

- **Lớp `VerticalAprioriModel`**:
  - **Xây dựng dữ liệu dọc (`_build_vertical_data`)**: Chuyển đổi dữ liệu từ dạng dòng (giao dịch) sang dạng cột (mặt hàng). Mỗi mặt hàng sẽ quản lý một tập hợp (set) các **Transaction IDs (TIDs)** mà nó xuất hiện. Điều này giúp việc tính Support chỉ đơn giản là đếm số lượng ID trong tập hợp.
  - **Hàm đệ quy (`_recurse`)**: Thay vì duyệt theo chiều rộng như thuật toán Apriori, Eclat duyệt theo chiều sâu. Thuật toán tìm các tập phổ biến bằng cách lấy **giao (intersection)** của các tập TID. Nếu kích thước tập giao thỏa mãn `min_support`, một tập phổ biến mới được hình thành.
- **Hàm `generate_rules_eclat`**:
  - Sử dụng kết quả từ tập phổ biến để tính toán **Confidence** (Độ tin cậy) sinh ra các luật kết hợp.
  - Công thức tính:

$$Confidence(A \to B) = \frac{Support(A \cup B)}{Support(A)}$$

2. So sánh kết quả giữa code thư viện và code tự viết

Dựa trên kết quả Output khi thực thi, chúng ta có bảng so sánh các chỉ số **Support** như sau:

Mặt hàng / Tập mục	pyECLAT	Code tự viết
Milk	0.772727	0.772727
Bread	0.727273	0.727273
Wine	0.727273	0.727273

Mặt hàng / Tập mục	pyECLAT	Code tự viết
Butter	0.681818	0.681818
Apple	0.681818	0.681818
Chips	0.636364	0.636364
Wine & Milk	0.636364	0.636364

Nhận xét:

1. **Về độ chính xác**: Kết quả tính toán Support của mã nguồn tự viết hoàn toàn trùng khớp với thư viện chuyên dụng `pyECLAT`. Điều này chứng minh logic xử lý tập hợp (intersection) và đếm TIDs đã được triển khai chính xác.
2. **Về luật kết hợp**: Phần code tự viết đã mở rộng thêm chức năng tìm luật kết hợp (Association Rules). Kết quả chỉ ra rằng khách hàng mua **Milk** có xác suất **82.35%** sẽ mua thêm **Wine**, và ngược lại, mua **Wine** có xác suất **87.5%** sẽ mua thêm **Milk.**
3. **Về hiệu năng**: Thư viện `pyECLAT` có thanh tiến trình và được tối ưu hóa cho các tập kết hợp lớn, trong khi code tự viết tập trung vào tính minh bạch của thuật toán đệ quy.

3. Kết luận

Việc tự triển khai thuật toán không chỉ giúp kiểm soát sâu hơn vào quá trình xử lý dữ liệu (đặc biệt là việc xử lý giá trị `NaN`) mà còn cho phép tùy biến linh hoạt việc sinh luật kết hợp (Confidence), điều mà hàm `fit` cơ bản của `pyECLAT` trong ví dụ trên chưa thể hiện rõ.

Phương pháp Vertical Eclat tự viết đã tận dụng tốt sức mạnh của phép toán tập hợp (Set) trong Python để cắt tỉa không gian tìm kiếm và tối ưu hóa việc tìm kiếm tập phổ biến.